

CDS View 課程練習 1：CDS View 是什麼、為什麼要用

Lecture

這門課要教什麼

歡迎來到 CDS View 課程。這門課不要求你先修過 RAP 課程 ([src/ABAP_Training_RAP/](#))，但如果你上過，會發現有些內容似曾相識——RAP 課程的 rap02 已經教過一部分 CDS View 基礎 (那是「為了銜接 Behavior Definition」的精簡版)。這門課要做的是把 **CDS View** 獨立出來，從零開始教一次完整體系：不只是 RAP 的地基，CDS View 本身就是分析型報表、Fiori Elements 畫面、甚至一般 Open SQL 查詢效能的核心技能。

這一課 (cds01) 的目標很單純：搞懂「CDS View 到底是什麼、解決什麼問題」，並且從零在 Eclipse ADT 建出你人生第一個 CDS View。

從「直接查表」的痛點說起

假設你要寫一支報表，需要查航空公司資料。最直覺的做法是直接對標準表 **SCARR** 下 Open SQL：

```
SELECT carrid, carrname, currcode
FROM scarr
INTO TABLE @DATA(lt_carriers).
```

這樣寫完全合法，也真的能跑。但這種「直接查表」的做法，隨著系統規模變大，會累積幾個實際的痛點：

1. **查詢邏輯到處複製貼上**：如果 10 支程式都需要「航空公司資料，只要啟用中的、換算成本地幣別顯示」這種稍微複雜一點的邏輯，每支程式各自寫一次 **WHERE** 條件、各自寫一次換算公式——邏輯散落各處，改一次要改 10 個地方，很容易漏改或改到不一致。
2. **技術欄位名稱、沒有語意標記**：**SCARR** 的欄位是 **CARRID** / **CARRNAME** / **CURRCODE** 這種技術命名，直接查表拿到的結果集**不會自動帶有**「這是金額」「這是數量，單位是什麼」這類語意資訊——這些資訊要靠程式自己額外處理。
3. **每次查詢都是從 Application Server 發一句新的 SQL 到資料庫**：如果查詢邏輯很複雜 (多層 Join、聚合)，沒有一個「共用、預先設計好」的查詢單元可以重複利用，效能調校的責任全部落在每一支呼叫端程式身上。

CDS View (Core Data Services View) 要解決的正是這個問題：把「查詢邏輯」本身當作一個可以命名、可以重用、可以疊層設計的 Repository 物件來管理，而不是散落在各支程式的 **SELECT** 陳述式裡。

CDS View 到底是什麼——三個核心特性

用一句話說：**CDS View** 是一個用宣告式語法 (**DDL**，**Data Definition Language**) 定義出來的、可以重複查詢的資料模型單元。這句話裡有三個關鍵字，分別對應三個核心特性：

特性	意思	對照上面提到的痛點
語意豐富化 (Semantically Rich)	CDS View 可以用 Annotation 幫欄位加上標籤、金額/數量的參考單位、存取權限規則等中繼資料，這些資訊會跟著這個 View 一起被任何消費端 (ABAP 程式、Fiori Elements、分析工具) 自動讀到	解決「沒有語意標記」的問題
可重用 (Reusable)	建一次、任何 ABAP 程式 / 其他 CDS View / Fiori Elements 都可以直接拿來查，查詢邏輯只維護在一個地方	解決「邏輯到處複製貼上」的問題
下推執行 (Push-down / Code to Data)	CDS View 本質上還是被編譯成資料庫可以執行的 SQL View，查詢時邏輯是在資料庫層執行，不是先把資料整批撈回 Application Server 才處理——這一點其實直接查表的 Open SQL 也做得到 (見下方澄清)，CDS View 的價值在於把這個下推的查詢邏輯做成一個可以重用、可以疊層設計的單元，而不是每支程式各自下推各自的 SQL	解決「沒有共用查詢單元」的問題

⚠ 一個容易誤會的地方要先澄清：「下推執行」不是 CDS View 專屬的能力——你在本課開頭那句直接查 SCARR 的 Open SQL，一樣是整句 SQL 送到資料庫執行、在資料庫端就篩選完畢，不會把整張表撈回 Application Server 才處理 (這是 Open SQL 的基本行為，從沒有 CDS 的年代就是這樣)。CDS View 並沒有讓「下推」這件事變得更快，它真正的價值是把這個下推的查詢邏輯變成一個可以重用、可以疊層設計的 **Repository 物件**——這一點會在 cds03 (Association vs. JOIN) 看到更具體的體現。

這一課的範例：把「直接查表」跟「查 CDS View」放在一起比較

這一課會建立一個最基本的 CDS View——**ZI_CDS01_CARRIER**，包在標準表 **SCARR** (航空公司主檔) 外面。先看 **SCARR** 本身的 DDL (欄位技術名稱，沒有任何額外語意)：

```
define table scarr {
  key mandt   : s_mandt not null ...
  key carrid  : s_carr_id not null;
  carrname    : s_carrname not null;
  currcode   : s_currcode not null ...
  url         : s_carrurl not null;
}
```

ZI_CDS01_CARRIER 的 DDL：

```
@AbapCatalog.sqlViewName: 'ZICDS01CARR'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS01: Airline Carrier Basic View'
define view ZI_CDS01_CARRIER
  as select from scarr
{
```

```
key carrid,  
carrname,  
currcode,  
url  
}
```

這一課先示範最基本的情況——欄位清單原封不動照抄，沒有改名、沒有計算欄位（那是 cds02 的主題），單純只是「把 SCARR 包一層」。你可能會問：這樣包一層，除了多一層間接，到底帶來什麼好處？老實說，這個最基本的版本，好處確實還看不太出來（`SELECT * FROM zi_cds01_carrier` 跟 `SELECT * FROM scarr` 幾乎沒有差別）——這是刻意的：這一課的重點是先搞懂 CDS View 的建立語法跟基本結構，好處會在後續課程逐一疊加上去才會真正顯現：cds02 會加計算欄位跟函數，cds03 會加 Association（這是直接查表完全做不到的），cds06 會加真正的權限控管邏輯。到了 cds08（期中整合），你會看到一個真正發揮出「重用、疊層設計」價值的完整案例。

⚠ 這系統的 CDS 編譯器不支援新式 `define view entity` 語法

在動手之前，有一個系統限制務必先知道，否則你會在第一次啟用就卡住：

SAP 官方教材（2020 年之後）預設用的是新式語法：

```
define root view entity ZI_XXX as select from ...
```

這個系統（On-Premise S/4HANA 1909）的 CDS 編譯器不認得 `entity` 這個關鍵字，用這種寫法啟用會直接報錯：

```
Syntax error: Keyword ENTITY not allowed
```

必須改用舊式語法（拿掉 `entity`）——也就是上面 `ZI_CDS01_CARRIER` 範例實際用的寫法：

```
define view ZI_XXX as select from ...
```

根本原因：這個系統的開發套件沒有啟用「ABAP Cloud 語言版本」（SAP 訂出的一個語言限制開關，S/4HANA 2022 以後的 On-Premise 系統，或 SAP BTP ABAP Environment 才能啟用），新式 CDS View Entity 語法是 ABAP Cloud 世代一起引進的語言特性，這個系統的套件沒有這個開關就用不了。這個限制已經在 RAP 課程完整查證並記錄過（[.claude/rules/sap-adt-mcp.md](#) 第 40 節），這門課直接沿用結論，之後每一課只要遇到 CDS View 語法，一律預設用這種舊式寫法，不用每次重新驗證。

這不是「查不到路徑」這種空殼限制——這系統的 CDS View 框架是真的在運作，只是語法版本比較舊。SAP 自己出的標準內容在這個系統上也是用這種舊式語法（例如標準物件 `C_SalesOrderManage`），不是只有我們自己建的物件才這樣。

Eclipse ADT 建立 CDS View：Step by Step

現在動手建 `ZI_CDS01_CARRIER`：

1. 在 **Project Explorer** 展開你的 ABAP Project，找到套件 `$TMP`（本課程一律用這個套件）。
2. 對著 `$TMP` 按滑鼠右鍵 → **New** → **Other ABAP Repository Object...**，篩選 `Data Definition`，選取後按 **Next**。

3. 填寫：

- **Name** : `ZI_CDS01_CARRIER`
- **Description** : `CDS01 Basic Interface View on SCARR`
- **Package** : `$TMP`
- 按 **Next** (`$TMP` 套件會跳出「Select Transport Request」畫面，顯示「No change recording enabled for package \$TMP」時，什麼都不用選，直接按 **Finish** 即可)

4. ⚠️ ⚠️ 接著會跳出「Templates」畫面，這一步選錯會直接撞上上一節講的語法限制：

| 模板 | 語法關鍵字 | 這系統能不能用 | |---|---|---| | Define View Entity | `define view entity X as select from Y` (新式，帶 `entity`) | ❌ 不能——啟用會直接失敗 | | Define Root View Entity | 同上，多一個 `root` 關鍵字 | ❌ 不能，同樣原因 | | Define View Entity with To-Parent Association | 同上，多預帶一段 `association to parent` 骨架 | ❌ 不能，同樣原因 | | **Define View (obsolete as of AS ABAP 7.57)** | `define view X as select from Y` (舊式，沒有 `entity`) | ✅ 選這個——雖然畫面標示「obsolete」，這系統的 ABAP 版本只吃這種舊式語法 |

選好「Define View (obsolete as of AS ABAP 7.57)」後，畫面下方會預覽這個模板的骨架內容，確認長得跟本課要寫的語法一致再按 **Next**。

1. 精靈通常會再問你要不要以某個既有 Table 當範本 (Reference Object)，選 `SCARR` 可以讓系統自動帶出一個包含全部欄位的初始 `SELECT` 清單，省去手動打欄位名稱。
2. 編輯器開啟後，把系統產生的骨架改成上面「這一課的範例」程式碼區塊列出的完整內容 (三個 annotation + 欄位清單)。
3. **Ctrl+S** 存檔 → 按工具列的 **Activate** (或 **Ctrl+F3**) 啟用。
4. 對著這個 View 按右鍵 → **Open With** → **Data Preview**，應該會直接看到 `SCARR` 現有的資料 (航空公司清單)，欄位標題會顯示標準 Data Element 的說明文字 (例如 `Airline`)。

三個最基本的 Annotation

```
@AbapCatalog.sqlViewName: 'ZICDS01CARR'
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'CDS01: Airline Carrier Basic View'
define view ZI_CDS01_CARRIER
  as select from scarr
{ ... }
```

`@AbapCatalog.sqlViewName`——DDL View Name 跟 SQL View Name 是兩個不同名字：

- **DDL View Name** (`define view ZI_CDS01_CARRIER` 這裡的名稱)：這是你在 Eclipse ADT / ABAP 程式碼裡實際引用這個 View 用的「邏輯名稱」，也是 Repository 物件的正式名稱，長度上限是一般 Z 物件的 **30 碼**。
- **SQL View Name** (annotation 指定的值，例如 `'ZICDS01CARR'`)：這是這個 View 在資料庫底層真正建出來的**實體 SQL VIEW 物件名稱**，官方文件明講這是「純技術用途的輔助物件」("purely technical helper construct")，長度上限只有 **16 碼**——這是這系統用的舊式 `define view ( V1 )` 語法**強制要求**的，一定要指定，不能省略。
- **為什麼要分兩個名字**：DDL View Name 為了可讀性可以取到 30 碼，但資料庫底層物件名稱上限比較短 (沿襲自 ABAP Dictionary 的命名規則)，所以要另外指定一個 ≤ 16 碼的版本給底層資料庫用。這一課的 `ZI_CDS01_CARRIER` (16 碼) 刪掉底線縮成 `ZICDS01CARR` (11 碼) 就是這樣來的。

@AccessControl.authorizationCheck：控制這個 View 要不要做權限檢查，這一課先用最簡單的 **#NOT_REQUIRED**（完全不檢查），讓這一課專心在「CDS View 怎麼建」這件事上；真正的權限控管語法（**#CHECK**、**define role**）是 cds06 的主題，那時候才會回頭把這個值換掉。

@EndUserText.label：這個 View 本身的說明文字，會出現在 Repository 物件清單、Where-Used 搜尋結果等地方，純粹是給開發者/管理者看的說明，不影響查詢邏輯。

SE11 / SE16 / Eclipse Data Preview：三種驗證查詢結果的方式

CDS View 不是可執行的程式，沒有「跑一下看結果」這種直覺的驗證方式，要靠下面幾個工具個別驗證：

① **Eclipse ADT Data Preview**（已用 ADT API 實測確認，**ZI_CDS01_CARRIER** 查得到 **SCARR** 的真實資料，欄位標題正確顯示 **Airline / Airline Currency** 這些標準 Data Element 的說明文字）：對著 View 按右鍵 → **Open With** → **Data Preview**，直接看到查詢結果，用 **DDL View Name**（**ZI_CDS01_CARRIER**）識別物件，這是 Eclipse ADT 這類「CDS 之後才有的現代化工具」的一貫行為。

② **SE11（View 顯示畫面）**：這裡有一個容易誤會的地方——**SE11** 反而是用 **SQL View Name**，不是 **DDL View Name**（這一點 RAP 課程 rap02 已經用使用者實測確認過，適用所有掛在 **define view**（V1）物件上的 CDS View，不是 **ZI_CDS01_CARRIER** 特有的狀況）：**SE11** → **View** → 輸入 **ZICDS01CARR**（SQL View Name，不是 **ZI_CDS01_CARRIER**）→ **Display**，查詢欄位本身會標示 **DDL SQL View**；查到之後畫面上另外會顯示一個 **DDL Source** 欄位，回頭告訴你這個底層 SQL View 是哪個 CDS DDL 定義產生的（顯示 **ZI_CDS01_CARRIER**）。**原因**：SE11 的 View 瀏覽功能是比 CDS 更早就存在的傳統 ABAP Dictionary 機制，本質上是直接對應資料庫底層那個物理物件在查，CDS View 只是後來「掛」進這套舊機制。

③ **SE16（資料瀏覽器）**——⚠️ 這一段是根據 **SE11** 已驗證過的規則類推，**Claude** 沒有 **SAP GUI** 可以親自操作 **SE16**，還沒有實際測試過：因為 **@AbapCatalog.sqlViewName** 建出來的 SQL View 是一個真正登記在 ABAP Dictionary 裡的資料庫物件（跟 SE11 View 畫面查到的是同一個物理物件），推測 SE16 應該同樣要輸入 SQL View Name（**ZICDS01CARR**）才查得到資料。這只是根據既有規則的合理推論，不是已經確認的結論——請你實際在 SE16 操作一次（**SE16** → 輸入 **ZICDS01CARR** → **Display**），把畫面實際看到的狀況回報給我，我會依照你的實測結果修正這一段說明（如果推論錯了、需要用別的名稱或方式才能查到，也一起告訴我）。

這一課學到的東西，接下來會怎麼用

- cds02：欄位怎麼改名、怎麼加計算欄位跟內建函數
- cds03：Association——CDS View 真正跟直接查表拉開差距的地方
- cds06：把 **@AccessControl.authorizationCheck: #NOT_REQUIRED** 換成真正的權限控管
- cds08（期中整合）：把 cds01~cds07 教的技巧疊成一個真正有價值的多層 CDS View

Eclipse ADT Step by Step（重點回顧）

1. 對著 **\$TMP** 套件右鍵 → **New** → **Other ABAP Repository Object** → **Data Definition**
2. 填 Name / Description / Package，**\$TMP** 的 Transport 畫面直接 Finish
3. **Templates** 畫面務必選「**Define View (obsolete as of AS ABAP 7.57)**」，不要選任何帶 **Entity** 字樣的模板
4. 選 **SCARR** 當 Reference Object，帶出初始欄位清單
5. 改成本課要求的完整內容（三個 annotation + 欄位清單）
6. **Ctrl+S** 存檔 → **Activate**
7. 右鍵 → **Open With** → **Data Preview**，確認查得到資料

學習目標

- 能講出「直接查表」的三個實際痛點（邏輯到處複製貼上、沒有語意標記、沒有共用查詢單元），以及 CDS View 對應解決的三個核心特性（語意豐富化、可重用、下推執行）
- 知道「下推執行」不是 CDS View 專屬的能力，Open SQL 直接查表本來就會下推，CDS View 的價值在於把下推的查詢邏輯做成可重用、可疊層設計的物件
- 能講出這個系統的 CDS 編譯器不支援新式 `define view entity` 語法的具體錯誤訊息與根本原因（套件沒有啟用 ABAP Cloud 語言版本）
- 能在 Eclipse ADT 完整走過一次「New Data Definition」精靈，從空殼建出一個基本 CDS View，並且在 Templates 畫面正確選擇「Define View (obsolete as of AS ABAP 7.57)」而不是任何帶 Entity 字樣的模板
- 能寫出三個最基本的 CDS annotation (`@AbapCatalog.sqlViewName` / `@AccessControl.authorizationCheck` / `@EndUserText.label`) 各自的作用
- 知道 DDL View Name（邏輯名稱，上限 30 碼，Eclipse / ABAP 程式碼用）跟 SQL View Name (`@AbapCatalog.sqlViewName` 指定，資料庫底層實體物件名稱，上限 16 碼) 是兩個不同名字
- 知道不同工具查詢 CDS View 要用哪個名字的判斷原則：CDS 之後才有的現代工具（Eclipse ADT、Open SQL）用 DDL Name；比 CDS 更早存在的傳統工具（SE11 View 畫面）用 SQL View Name
- 能用 Open SQL 分別查詢一張標準表跟包住它的 CDS View，驗證兩者查詢結果一致

物件清單

物件	名稱	型別
CDS Interface View	<code>ZI_CDS01_CARRIER</code>	DDLS/DF
驗證程式	<code>ZR_CDS01_DEMO</code>	PROG/P

全部物件都在 `$TMP` 套件，已建立並啟用（`sap_inactive_objects` 確認 0 筆殘留）。

動手練習物件（你自己動手建，名稱自訂，不算在上面正式的課程物件清單裡）：

物件	建議名稱	型別	對應練習
CDS View（基於 SPFLI）	<code>ZI_CDS01_FLIGHT_BASIC</code> （自訂）	DDLS/DF	動手練習

動手練習

輪到你了：用標準示範資料表 `SPFLI`（航班基本資料）自己建一個全新的基本 CDS View（物件名稱、套件 `$TMP`，命名自訂，例如 `ZI_CDS01_FLIGHT_BASIC`），要求：

1. 欄位清單至少包含：`carrid`（key）、`connid`（key）、`cityfrom`、`cityto`——這一課先不要加 **Association**、不要加計算欄位，單純練習「基本 CDS View 的建立語法」，這些進階功能留到後面幾課
2. 三個基本 annotation 都要寫（`sqlViewName` / `authorizationCheck` / `EndUserText.label`），SQL View Name 自己想一個 ≤16 碼、不跟系統裡既有物件衝突的名稱
3. 用 Data Preview 確認查得到資料（`SPFLI` 標準系統應該有數十筆示範資料）

建好、啟用成功後跟我說一聲（貼程式碼或截圖都可以），我會幫你核對語法有沒有踩到這個系統的已知坑（例如 `entity` 關鍵字、SQL View Name 超過 16 碼）。

如果你想額外挑戰一下：試著故意把 Templates 畫面改選成「Define View Entity」，實際看一次啟用失敗的錯誤訊息長什麼樣子，再改回正確的模板重建一次——親眼看過這個錯誤，比只是讀講義印象更深。

驗證方式

CDS View 不是可執行的程式，沒有 `programrun` 這種無頭驗證手段（除非透過一支呼叫它的 ABAP 程式間接驗證，見下方第 1 點），這一課的驗證重點是語法正確 + 成功啟用 + 查得到資料：

1. 已用 `ZR_CDS01_DEMO` 透過 `programrun` 無頭驗證：同一個航空公司清單，分別用 Open SQL 直接查 `SCARR` 跟查 `ZI_CDS01_CARRIER` (DDL Name)，兩邊筆數一致，`ZI_CDS01_CARRIER` 額外正確查得到 `url` 欄位，證實 CDS View 查詢結果跟底層表資料一致：

```
``text === 1. 直接查表 SCARR (Open SQL) === AA American Airlines USD AB Air Berlin EUR AC
Air Canada CAD === 2. 查 CDS View ZI_CDS01_CARRIER (DDL Name) === AA American Airlines
USD http://www.aa.com AB Air Berlin EUR http://www.airberlin_obsolete.de AC Air Canada
CAD http://www.aircanada.ca === 3. 筆數是否一致 === MATCH: row counts equal 3 ``
```

1. `sap_inactive_objects` 回傳空清單，代表沒有殘留的未啟用版本
2. 已用 ADT Data Preview API 直接確認 `ZI_CDS01_CARRIER` 查得到 `SCARR` 的真實資料（航空公司清單），欄位標題正確顯示標準 Data Element 說明文字

動手練習的驗證方式：Eclipse 啟用成功 + 用 Data Preview 看查詢結果，貼程式碼給我核對語法。

思考題

1. 這一課的 `ZI_CDS01_CARRIER` 欄位清單跟底層表 `SCARR` 幾乎一模一樣（只是重新列了一次），如果一支程式現在就想拿這個 View 的資料，跟直接查 `SCARR` 相比，實際上有什麼差別？（提示：回顧「CDS View 到底是什麼」那一節，誠實想一下——這一課的版本真的有帶來什麼立即的好處嗎？）
2. `@AccessControl.authorizationCheck: #NOT_REQUIRED` 代表完全不做權限檢查。如果這個 View 之後要曝光給外部系統（例如透過 Fiori Elements），一直維持 `#NOT_REQUIRED` 會有什麼風險？
3. 如果你把 `@AbapCatalog.sqlViewName` 的值故意取超過 16 碼（例如直接照抄 DDL Name `ZI_CDS01_CARRIER`，剛好 16 碼；如果再長一點呢），啟用時你預期會發生什麼事？
4. 承第 3 題，如果系統裡已經有另一個 CDS View 的 SQL View Name 剛好跟你取的一樣（例如兩個不同的 DDL View 都想用 `ZICDS01CARR` 當 SQL View Name），你覺得系統會怎麼處理？（提示：SQL View Name 對應的是資料庫底層真正的實體物件名稱，資料庫裡同名的物件只能有一個）
5. SE11 查 CDS View 要用 SQL View Name，Eclipse ADT / Open SQL 要用 DDL View Name——如果你只知道其中一個名字，要怎麼查出另一個？（提示：回顧「SE11」那一節提到的 DDL Source 欄位）

答案

見 `zi_cds01_carrier.ddls.abap`、`zr_cds01_demo.prog.abap`。SAP 端物件：`ZI_CDS01_CARRIER`（CDS View）、`ZR_CDS01_DEMO`（驗證程式）。動手練習（基於 `SPFLI` 的基本 CDS View）由你在 Eclipse 動手建立，沒有固定答案快照——建好後跟我核對即可。